



How Machine Learning can Enhance Art

Machine learning is everywhere and the field of art is no exception. This article explores applications of machine learning in painting and music.

Machine learning has emerged as an important and powerful tool for software developers. It has helped solve problems that were earlier considered unsolvable, and has been adopted in a wide spectrum of domains. In addition to the various tech domains that have benefited from ML, many art forms have also benefited from it. This article focuses on two major art forms — music and painting.

Various tools can assist in applying ML to the creative process. Some of them are:

- Magenta
- Google Deep Dream
- MuseNet

There are various applications that have been developed using popular tools such as TensorFlow. This article has two major sections:

- The first section explores neural style transfer using TensorFlow.
- The second section explores Magenta and its application to music.

Neural style transfer

Composing one image in the style of another image is called neural style transfer. Using this technique, an ordinary picture can be converted into a style of painting created by the masters such as Picasso, Van Gogh or Monet. The major steps in neural style transfer are:

- Get an image

- Get a reference image
- Blend the images with ML
- Output the image with the transformed style

Figure 1 shows a source image (the banner image of an earlier article titled, 'Is your website accessible and inclusive?' in the March 2020 issue of *OSFY* available at <https://opensourceforu.com/2020/03/is-your-website-accessible-and-inclusive/>) and a reference style image (a Kandinsky painting).

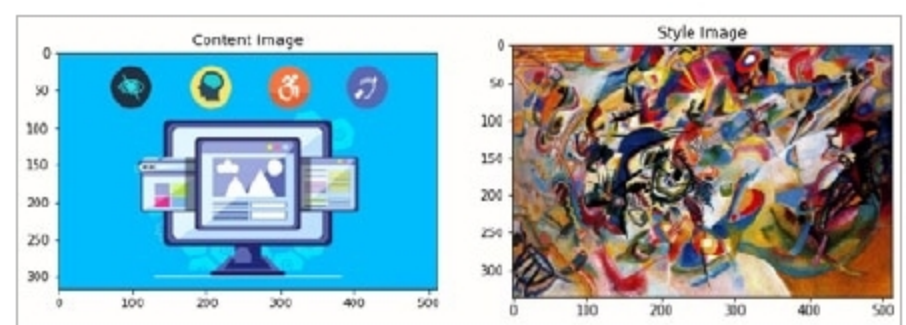


Figure 1: A source image and a style image

The fast style transfer using TensorFlow Hub is shown in Figure 2.

The code snippet with TensorFlow Hub is as follows:

```
import tensorflow_hub as hub
hub_module = hub.load('https://tfhub.dev/google/magenta/
arbitrary-image-stylization-v1-256/1')
stylized_image = hub_module(tf.constant(content_image),
tf.constant(style_image))[0]
tensor_to_image(stylized_image)
```

